

MONSTRUMS GAMING · STAKE ENGINE

TAP TRADE

Tap-cell-to-bet on a live price chart. A direct-probability prediction arcade built on the certified-replay Stake Engine RGS — 28 win/lose multiplier modes, 1.4x to 100x, RTP 96.00–96.55%.

DOCUMENT	Game Design Document — internal
GAME ID	2_6_tap_trade (math) · 2-6-tap-trade (web)
VERSION	1.0 · 2026-07-22
MATH	28 modes · RTP 96.00–96.55% · max win 100x
STATUS	Feature-complete · pre-ACP-upload

Monstrums-Gaming/math-sdk · games/2_6_tap_trade – Monstrums-Gaming/monstrums-web-sdk · apps/2-6-tap-trade

§ Contents

1	Overview	concept, platform, lineage
2	Core Gameplay	loop, grid, placement rules, the 2-chip cap
3	Game Math	ladder derivation, full odds table, risk envelope
4	Presentation & UX	HUD, themes, history, settings, mobile
5	Fairness & Compliance	certified replay, ACP rules, jurisdiction
6	Technical Architecture	book contract, tap queue, masked ledger
7	Status & Roadmap	done, launch gates, later
8	Appendix	file map, storage keys, glossary

1 Overview

Tap Trade is a continuous, tap-to-bet prediction game (the “Euphoria” mechanic). A live-looking price chart scrolls forever; the future region of the chart is covered by a grid of (*time* × *price*) multiplier cells. The player taps a cell to place a chip. If the price line reaches that cell, the chip pays $\text{bet} \times \text{cellMultiplier}$; otherwise it loses. There are no rounds and no timers — each chip resolves the moment the line reaches its column.

Genre	Direct-probability prediction arcade (crash-adjacent, per-decision wagers)
Platform	Stake Engine (certified-replay RGS, iframe launch with <code>rgs_url</code> + <code>sessionId</code>)
Session shape	Continuous; multiple concurrent chips; a chip = one independent wager
Bet range	Operator bet-level grid (offline reference grid \$1–\$1,000)
Max win	100× (top ladder rung, Limbo <code>base_100</code> precedent)
Math model	Each chip is an independent win/lose bet at a fixed multiplier M — a <code>2_9_crypto_pulse</code> twin
Lineage	<code>2_11_price_grid</code> → <code>2_11_crypto_pulse_grid</code> (28-rung rebuild; the 20-rung <code>2_10</code> was retired) → renamed <code>2_6_tap_trade</code> to pair with web <code>apps/2-6-tap-trade</code>

The one-sentence pitch: the thrill of calling a price move, priced like a certified casino game — every cell on screen is a real published bet mode, and the chart is steered theatre over pre-generated, hash-frozen outcomes.

2 Core Gameplay

2.1 The loop

1. **Tap a future cell.** Each cell displays a multiplier snapped to the published ladder. A chip is placed for the current bet size (one real `/wallet/play` at that cell's mode).
2. **The line approaches.** The chip pulses *pending* → *active* → *hot* (with a depleting countdown bar) as its column nears the now-line.
3. **Reveal.** The line is steered to hit (win) or miss (lose) the cell per the drawn book. A win fills the cell gold with a ring pulse and pays $\text{stake} \times M$; a loss flashes red, shakes, and drops away.

2.2 Grid geometry

Cell size	5 s wide × \$0.50 tall (14 px/s × 110 px/\$ at zoom 1)
Now-line	At 40% of the canvas width; history scrolls left of it
Dead zone	Cells starting < 2.5 s ahead cannot be bet (shaded)
Reveal window	A chip resolves within 1.6 s of its column reaching the now-line
Zoom	0.5×–2.5× (wheel / pinch / buttons) – rescales pixels only, never odds

2.3 Placement rules

- Cell must start ≥ 2.5 s in the future ("Too close to the line").
- One chip per cell ("Chip already there").
- **Max two chips per time-column**, on distinct cells ("Column full").
- Stake must be $\leq$ the displayed balance (queued chips reserve funds).
- Chips in *other* columns are unrestricted — their rounds are serialized through the bet queue, not rejected.
- **Tap-to-confirm** (setting; default on for touch): first tap previews the cell and its exact offer, second tap places.

Why exactly two chips per column. During a column's reveal window the line sweeps one *contiguous* band of prices, so the column's winners must be adjacent in price with every loser outside the band. With 2 chips all 4 outcome patterns are drawable (sweep both, hit either, thread between). With 3 chips a *win-lose-win sandwich* (1 of 8 draws) is impossible to render; with 4 chips ~31% of patterns are. Payouts would still be correct — but the line would visibly contradict a result. The cap is **static** so a rejection never leaks anything about outcomes already drawn on the board.

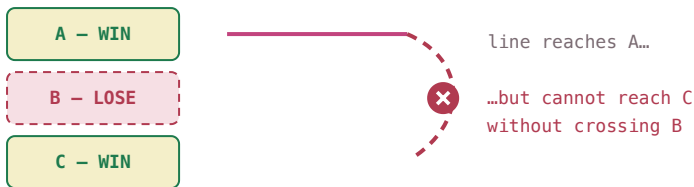


FIG. 1 – THE UNRENDERABLE WIN-LOSE-WIN SANDWICH THAT MOTIVATES THE 2-CHIP COLUMN CAP

3 Game Math

3.1 Model

Tap Trade is a **direct-probability** game: no board, no reels, no free spins, Rust optimiser off. Every distinct cell multiplier is its own published bet mode `call_<cents>` (cost 1.0). Lookup-table weights are uniform (1 per book), so **published odds equal book counts exactly** — there is no approximation anywhere in the pipeline.

3.2 Ladder derivation

For each multiplier M, the win probability is the *smallest-denominator* rational a/b whose realised RTP $(a/b) \times M$ lands in $[96.00\%, 96.70\%]$ (the `_simplest_fraction_in` Stern-Brocot descent shared with Limbo/Chicken). `num_sims = b`, so a build yields exactly a winning books out of b — the LUT is the odds.

Why the floor is 1.4x, not 1.2x: a 1.2x win/lose mode has payout standard deviation ≈ 0.48 , below ACP's Base-Volatility floor of 0.60 — which rates the whole game off its tamest mode. 1.4x clears it (~ 0.62). The 100x ceiling is the approved Limbo `base_100` precedent; the whole ladder sits inside that envelope at both ends. Every mode is a two-outcome all-or-nothing bet.

3.3 The 28-rung odds table

Generated from the verified odds bundle (`build_odds_bundle.py` re-checks every published LUT row and book event before emitting it). Band 96.00–96.55%, cross-mode spread 0.5517% ($\leq 1\%$ required), 1,176 books total, largest mode 135 books.

MODE	MULTIPLIER	WIN CHANCE (A/B)	WIN %	RTP	BOOKS
call_140	1.4x	11/16	68.75%	96.25%	16
call_150	1.5x	9/14	64.29%	96.43%	14
call_160	1.6x	3/5	60.00%	96.00%	5
call_180	1.8x	8/15	53.33%	96.00%	15
call_200	2x	12/25	48.00%	96.00%	25
call_220	2.2x	7/16	43.75%	96.25%	16
call_250	2.5x	5/13	38.46%	96.15%	13

MODE	MULTIPLIER	WIN CHANCE (A/B)	WIN %	RTP	BOOKS
call_280	2.8×	10/29	34.48%	96.55%	29
call_320	3.2×	3/10	30.00%	96.00%	10
call_360	3.6×	4/15	26.67%	96.00%	15
call_400	4×	6/25	24.00%	96.00%	25
call_450	4.5×	3/14	21.43%	96.43%	14
call_500	5×	5/26	19.23%	96.15%	26
call_600	6×	4/25	16.00%	96.00%	25
call_700	7×	4/29	13.79%	96.55%	29
call_800	8×	3/25	12.00%	96.00%	25
call_900	9×	3/28	10.71%	96.43%	28
call_1000	10×	5/52	9.62%	96.15%	52
call_1200	12×	2/25	8.00%	96.00%	25
call_1500	15×	5/78	6.41%	96.15%	78
call_2000	20×	4/83	4.82%	96.39%	83
call_2500	25×	1/26	3.85%	96.15%	26
call_3000	30×	4/125	3.20%	96.00%	125
call_4000	40×	2/83	2.41%	96.39%	83
call_5000	50×	1/52	1.92%	96.15%	52
call_6500	65×	2/135	1.48%	96.30%	135
call_8000	80×	1/83	1.20%	96.39%	83
call_10000	100×	1/104	0.96%	96.15%	104

3.4 The per-mode wincap quirk (intentional)

Each mode's `BetMode.max_win` equals its *own* multiplier M (not the global 100×). The engine applies a per-mode wincap override during sims, so a winning book — whose payout equals M — always reaches the active mode's cap and emits a standard `wincap` event. **Every winning book in every mode** is therefore `cellCall → wincap → finalWin`; a losing book is `cellCall → finalWin`. The web client treats `wincap` as an explicit no-op. Do not “fix” this.

3.5 Display pricing vs true odds

The multiplier a cell *shows* comes from a local volatility model (normal-CDF band probability × 1.55, house-edge 3.5%) snapped in log-space to the nearest ladder rung — so every offered value is a real published mode. This model is presentation only: the true odds always come from the mode the chip plays.

4 Presentation & UX

4.1 Screens & flow

- **Splash:** one loop of the Stake Engine loader (animated webp) on a #041721 ground → loader fades → ground eases to black → the Monstrums sting video fades in with audio. Autoplay ladder: sound → muted → skip. Tap (once playing) or Escape skips; a 20 s failsafe guarantees entry. The game boots underneath.
- **Main screen:** full-viewport canvas chart + floating chrome.

4.2 HUD map

Top-left	Juice wordmark (theme-aware inline SVG) · balance pill with LOCAL/LIVE badge
Top-right	Session strip pill: net P/L · amount in play · last-8 result pips (click → history)
Center	First-visit hint (fades 0.8 s after the first chip; persisted)
Mid-left	Zoom -/+ (docks to a vertical rail on phones)
Bottom-center	Tap Trade title badge (theme-aware; hidden on phones)
Bottom-right	Bet row: quick chips · custom slot · "+" grid menu · history · settings · theme

4.3 Bet picker

The RGS bet-level grid is authoritative (off-grid `/wallet/play` amounts are rejected with `ERR_VAL`). Quick chips are the design amounts `[1, 2, 5, 20]` snapped to the nearest grid values; the "+" menu offers the full grid with the top value styled **MAX**; a non-quick pick fills a single custom slot. Selection persists (`taptrade.betSize`). Every label is currency-aware (Intl via `?currency` + `?lang`, crypto-code fallback).

4.4 Settings (gear)

SETTING	OPTIONS	NOTES
Game speed	1× / 1.5× / 2×	Multiplies the sim clock only (base pace 1.3×); odds untouched; hidden above 1× under <code>disabledTurbo</code>
Sound	on / off	Synth cues (no audio assets): place tick, loss thud, win arpeggio scaled by tier
Tap to confirm	on / off	Pointer-agnostic two-tap placement; defaults on for touch, off for mouse

4.5 Themes

Seven complete palettes (canvas + CSS variables) switchable at runtime; persisted.



4.6 Bet history & replay

Every resolved chip keeps a **JPEG landing shot** — a crop of the live canvas around the cell captured ~0.5 s after the reveal beat, so the win fill / loss flash is visible — plus the line’s approach path. The panel (clock button, or the strip pips) lists the last 24 with thumbnails; an entry opens a modal that **replays the approach** on a faithful mini-chart (real cell grid at the live chart’s seconds-to-dollars aspect; the slice ends exactly where the shot shows). In-memory only — shots are too large for localStorage.

4.7 Feedback & celebration

EVENT	FEEDBACK
Win < 5×	Gold fill, ring pulse, float “+amount”, toast, 2-note arpeggio
Win 5–20×	Bigger ring/float, 3-note arpeggio
Win ≥ 20×	Widest ring, screen-edge vignette flush, 4-note arpeggio
Loss	Red flash + shake, chip drops away, float “Missed”, thud — <i>no</i> balance change (stake left at placement)
Chip states	pending (pulse) → active → hot (countdown bar) → won/lost

4.8 Mobile (≤ 700 px)

Full-width bottom bet bar (label hidden, ≥44 px targets, safe-area inset), vertically docked zoom rail at mid-left, compact brand + pill with the session strip wrapping below, edge-to-edge menus capped at 48 vh, title badge hidden. Pinch zoom with a ghost-tap guard. All motion respects `prefers-reduced-motion`.

5 Fairness & Compliance

5.1 Certified replay

The Stake Engine RGS is a certified **replay** system: books and lookup tables are pre-generated, hash-verified, and frozen at publish time. Odds cannot change at runtime. The platform owns the provably-fair seed pair; its HMAC selects which pre-generated book a round serves.

5.2 Position-neutral books

The book encodes only win/lose and the offered multiplier. *Where* the tapped cell sits is pure client-side presentation — the chart is steered to hit or miss the player's cell from the drawn outcome. Prices are RNG theatre; money is book-driven.

5.3 ACP rules satisfied

- 0.1x LUT grid — every multiplier is a multiple of 0.10.
- Per-mode RTP within [90%, 96.70%] — realised 96.00–96.55%.
- Cross-mode spread $\leq 1.00\%$ — realised 0.5517%.
- Base bet mode cost = 1.0 (all 28 modes).
- Risk/star-rating: all-or-nothing two-outcome modes inside the approved Limbo 1.40x–100x envelope (the one gate not provable locally).

5.4 Regulated-market behavior

<code>jurisdiction.disabledTurbo</code>	Hides the 1.5x/2x speeds; resets a persisted fast speed to 1x
<code>jurisdiction.socialCasino</code>	Relabels the bet row "Play"
Demo gating	LOCAL play-money runs only in dev builds or behind <code>?demo=1</code> ; a paramless production launch shows a blocking "launch from the casino" overlay
Fatal errors	<code>ERR_IS</code> / <code>ERR_ATE</code> raise a non-dismissible Reload modal; reload re-authenticates and resume settles any orphan round
Currency / locale	All amounts via Intl from <code>?currency</code> + <code>?lang</code> ; crypto codes fall back to a code-prefix format

Open launch gate: `provider_number` in `game_config.py` is a placeholder (2). Set the ACP-assigned value and re-run the full prod build before the upload that ships. Bet levels are configured in the ACP dashboard (bet-level template; `us_` prefix for Stake US), not emitted by the math.

6 Technical Architecture

6.1 Two repositories, one contract

```
math-sdk  games/2_6_tap_trade          # authors odds, simulates, emits books + LUTs
  run.py → create_books → generate_configs → execute_all_tests
  build_odds_bundle.py → library/odds_bundle.json  # re-verifies artifacts; feeds the client
  ladder
  frontend_demo/                             # Vite demo – the living reference spec for the web
  app

monstrums-web-sdk  apps/2-6-tap-trade    # the production frontend (Svelte 5, dev :3026)
  src/game/...  src/components/...      # consumes books via the shared actor → handler
  pipeline
```

6.2 The book contract (verified against published books)

```
win:  cellCall {result:"Win", isWin:true, payoutMultiplier:<M×100>, winChance:a/b}
      → wincap {amount:<M×100>}          # every winning book – client no-op
      → finalWin{amount:<M×100>}
lose: cellCall {result:"Lose", isWin:false, payoutMultiplier:<M×100>, winChance:a/b}
      → finalWin{amount:0}
```

Mode keys are `call_<multiplierCents>`; the client ladder derives from `odds_bundle.json` so math naming wins.

6.3 Money scales (three different ones — the classic trap)

FIELD	SCALE	EXAMPLE (\$5 BET AT 4.5×)
API amounts (<code>round.amount/payout</code> , balances)	× 1,000,000	5000000 / 22500000
Book event amounts (<code>cellCall.payoutMultiplier</code> , <code>wincap/finalWin.amount</code>)	× 100 (cents of bet)	450
<code>round.payoutMultiplier</code>	plain multiplier	4.5 – dividing by 100 silently pays 1/100th

6.4 Round lifecycle

`/wallet/authenticate` → balance, bet levels, jurisdiction, resumable round · `/wallet/play` (mode, $\times 1e6$ amount) → book events under `round.state` · **a loss settles inside the play response** (`round.active:false`); only a **win** calls `/wallet/end-round` (the shared machine's *singleRoundWin* path calls it eagerly and applies the balance after the play phase). One active round per session.

6.5 Multi-chip over a one-round machine: the tap queue

The shared xstate actor accepts `BET` only in `idle`. Taps therefore queue app-side (`game/tapQueue.svelte.ts`) and pump serially: each chip sets its own `activeBetModeKey / betAmount` just before dispatch; the `cellCall` handler does **not** await the reveal animation (opposite of crypto-pulse), so a round settles in ~1 RTT and the queue drains while the line is still travelling.

6.6 The masked balance ledger

Server balances (including a win's end-round credit) land at *settlement*, before the line reaches the chip. The HUD therefore reads a display ledger: `place = -stake · win reveal = +payout · loss = no change`, and reconciles to the authoritative `stateBet.balanceAmount` only when nothing is queued, in flight, or unrevealed. Per-round balance snapshots must never drive the pill — out-of-order reveals drift it by whole stakes (the “double-deduction” bug class).

6.7 Scene engine & steering

Raw 2D canvas, one rAF loop, fixed-tick accumulator (`tick 0.1 s × timeScale 1.3 × speed` setting; hidden-tab backlog is dropped so chips never resolve off-screen). Steering per nearest unresolved column: winners — converge on the closest unrevealed winner, then sweep to the other; losers — hold a miss lane clear of every losing band; inside the reveal window the outcome is hard-guaranteed with a margin clamp so a losing cell is never touched.

Svelte 5 lesson (hard-won): object references fork across `$state` proxy trees. A chip (`tapState`) holding a reference to its history entry (`historyState`) ended up mutating a *different* object than the panel rendered. Cross-store links must be **by id**, resolved through the owning store. Also: never key an `{#each}` by `Date.now()` — two chips in one column resolve in the same sim tick.

6.8 Offline & debug

`mockRgs.ts` monkey-patches `fetch` (dev or `?demo=1` only) with live-faithful settlement — a win stays `active` until end-round, so the real machine path runs offline. Debug handles (dev builds only): `window.__tt` (sim time, cfg, `tapState`), `window.__ttHistory`; the demo exposes `window.__cpg`.

7 Status & Roadmap

7.1 Done (as of 2026-07-22)

- Math: production build verified — 28 modes, SHA-256 + payout-hash checks green, `build_odds_bundle.py` re-verification passes, 13 engine tests.
- Demo (`frontend_demo/`, Vite): feature-complete — history + landing shots + replay, splash, settings, production hardening; its own vitest suite + CI.
- Web app (`apps/2-6-tap-trade`): full demo parity — tap queue, masked ledger, history, splash, settings, jurisdiction, currency, demo gating, mobile layout; 22 unit tests; verified against the built bundle.

7.2 Launch gates

1. Set the ACP-assigned `provider_number`; re-run the prod chain (`run.py` → `build_odds_bundle.py` → demo data).
2. ACP upload (`index.json`, `books_*.jsonl.zst`, `lookUpTable_*.csv`) + bet-level template in the dashboard.
3. LIVE smoke test on a real session: resume, multi-chip, balance-vs-RGS lockstep.

7.3 Later

- Autobet — “repeat my last tap” loop over `tapQueue.waitForReveal()` (the placement-offset hook already records `stateGame.lastOffset`).
- `/wallet/end-round` retry with backoff — lives in shared `utils-xstate`; reload + resume covers the stuck-win case meanwhile.
- Storybook interaction tests for the bet picker and history panel.

8 Appendix

8.1 Key files

```
math-sdk/games/2_6_tap_trade/
  game_config.py      # ladder, BetModes (max_win = own M), _simplest_fraction_in,
                       _validate
  game_events.py      # cell_call_event
  build_odds_bundle.py # post-build verifier + odds_bundle.json
  frontend_demo/      # Vite reference client (src/main.js, splash.js, style.css ...)

apps/2-6-tap-trade/src/
  game/tapQueue.svelte.ts      # tap queue + masked ledger (core design)
  game/bookEventHandlerMap.ts  # cellCall / wincap / finalWin
  game/history.svelte.ts      # landing shots + entries (link by id!)
  game/{betGrid,money,themes,constants}.ts
  components/Scene.svelte     # canvas engine (steering, zoom, capture)
  components/{ControlPanel,HistoryPanel,SessionStrip,Splash,...}.svelte
```

8.2 localStorage keys

taptrade.betSize · taptrade.theme · taptrade.muted · taptrade.seenHint ·
taptrade.speed · taptrade.tapConfirm

8.3 Glossary

TERM	MEANING
Book	Pre-generated per-round event sequence served by the RGS
LUT	Lookup table — one row per book with its payout; row counts are the odds
Rung / mode	One ladder multiplier = one published bet mode <code>call_<cents></code>
Chip	One placed bet on one cell — one RGS round
Reveal	The visual moment the line resolves a chip (ledger effects land here)
Reconcile	Snapping the display balance to the server truth at quiescence
Landing shot	JPEG crop of the canvas around a resolved cell for the history panel

8.4 Version history

VERSION	DATE	NOTES
1.0	2026-07-22	Initial GDD — game feature-complete, pre-ACP-upload